

E07 — Number of Islands (BFS / DFS)

Experiment: 7 | LeetCode: #200 | CO: CO2, CO3 | BT Level: BT5

Problem Statement

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are surrounded by water.

Example 1:

Input :
11110
11010
11000
00000

Output: 1

Example 2:

Input :
11000
11000
00100
00011

Output: 3

Concept: Grid as Graph

Each cell (r, c) is a graph node. Edges connect horizontally and vertically adjacent land cells. An island = a connected component of land cells.

Strategy: Count connected components using DFS or BFS, marking visited cells to avoid recounting.

Solution 1: DFS (Flood Fill)

```
def numIslands_dfs(grid):  
    """  
    DFS flood fill - sink visited land cells.  
    Time: O(m × n) | Space: O(m × n) recursion stack  
    """
```

```

if not grid or not grid[0]:
    return 0

rows, cols = len(grid), len(grid[0])
count = 0

def dfs(r, c):
    # Base case: out of bounds or water or already visited
    if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
        return

    grid[r][c] = '#'    # Mark as visited (sink the island)

    dfs(r + 1, c)
    dfs(r - 1, c)
    dfs(r, c + 1)
    dfs(r, c - 1)

for r in range(rows):
    for c in range(cols):
        if grid[r][c] == '1':
            count += 1
            dfs(r, c)    # Sink entire island

return count

```

Solution 2: BFS (Queue-Based)

```

from collections import deque

def numIslands_bfs(grid):
    """
    BFS flood fill.
    Time: O(m × n)    |    Space: O(min(m, n)) – BFS queue width
    """
    if not grid or not grid[0]:
        return 0

    rows, cols = len(grid), len(grid[0])
    count = 0

    def bfs(r, c):
        queue = deque([(r, c)])
        grid[r][c] = '#'    # Mark immediately to prevent re-queuing

        while queue:
            row, col = queue.popleft()
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                nr, nc = row + dr, col + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '1':
                    grid[nr][nc] = '#'

```

```

        queue.append((nr, nc))

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                bfs(r, c)

    return count

```

Detailed Trace

Grid (Example 1):

```

11110
11010
11000
00000

```

DFS starting at (0,0):

```
count = 0
```

```
(0,0) = '1' → count=1, start DFS from (0,0)
```

```

DFS(0,0):
    mark (0,0) = '#'
    DFS(1,0): mark '#'; DFS(2,0): mark '#'; DFS(3,0)='0' return
                DFS(2,1): mark '#'; DFS(2,2): mark '#'; DFS(2,3)='0' return
                DFS(1,1): mark '#'; DFS(0,1): mark '#'; DFS(0,2): mark '#'
                DFS(0,3): mark '#'; DFS(1,3): already '#' return
                DFS(1,2)='0' return

```

```
All land connected to (0,0) sunk as '#'
```

```
Remaining grid:
```

```

#####0
#####0
#####0
00000

```

```
(all 1s are now '#')
```

```
Continue scan: no more '1' found
```

```
count = 1 ✓
```

Grid (Example 2):

```

11000
11000
00100

```

00011

Step	r	c	Cell	Action
1	0	0	'1'	count=1, DFS sinks {(0,0),(0,1),(1,0),(1,1)}
2	0..1	2..4	'#' or '0' skip	
3	2	2	'1'	count=2, DFS sinks {(2,2)}
4	3	3	'1'	count=3, DFS sinks {(3,3),(3,4)}

Result: 3 ✓

Non-Destructive Variant (using visited set)

```
def numIslands_visited(grid):
    """Non-destructive: use visited set instead of modifying grid."""
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    visited = set()
    count = 0

    def dfs(r, c):
        if (r, c) in visited or r < 0 or r >= rows or c < 0 or c >= cols:
            return
        if grid[r][c] == '0':
            return

        visited.add((r, c))
        dfs(r + 1, c); dfs(r - 1, c)
        dfs(r, c + 1); dfs(r, c - 1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                count += 1
                dfs(r, c)

    return count
```

Test Suite

```
def test_num_islands():
    import copy
    test_cases = [
        ([["1", "1", "1", "1", "0"],
          ["1", "1", "0", "1", "0"],
          ["1", "1", "0", "0", "0"],
          ["0", "0", "0", "0", "0"]], 1),
        ([["1", "1", "0", "0", "0"],
```

```

        ["1", "1", "0", "0", "0"],
        ["0", "0", "1", "0", "0"],
        ["0", "0", "0", "1", "1"]], 3),
    ([["0"]], 0),
    ([["1"]], 1),
    ([["1", "0", "1"],
        ["0", "1", "0"],
        ["1", "0", "1"]], 5),    # Diagonal NOT connected
]

for grid, expected in test_cases:
    r1 = numIslands_dfs(copy.deepcopy(grid))
    r2 = numIslands_bfs(copy.deepcopy(grid))
    for r, name in [(r1, "dfs"), (r2, "bfs")]:
        status = "PASS" if r == expected else "FAIL"
        print(f"{status} [{name}]: {r} islands (expected {expected})

test_num_islands()

```

Complexity Analysis

Solution	Time	Space	Notes
DFS (recursive)	$O(m \times n)$	$O(m \times n)$ stack	Simple, risk of stack overflow on large grids
BFS	$O(m \times n)$	$O(\min(m, n))$	Safer for large grids
Non-destructive	$O(m \times n)$	$O(m \times n)$ visited	Preserves grid, extra memory

Key Takeaways

- **Grid traversal = graph traversal** — rows/columns map to graph vertices.
 - Both DFS and BFS work; BFS is preferred for large inputs due to bounded queue size.
 - **Mark immediately when queuing** (not when dequeuing) to avoid duplicates.
 - Diagonal cells are NOT adjacent in this problem — only 4-directional connectivity.
-

References

- LeetCode #200 — Number of Islands
- L31.md (BFS) and L32.md (DFS) — graph traversal theory
- Skiena, *The Algorithm Design Manual*, Ch. 7.6 (Connected Components)